The RX Path

How the node's receive audio gets from Asterisk into Node Talker, through the frame queue, the cleanup stage, and out to each listener, and what happens when the queue runs behind.

FEED HOOK · FRAMES · CLEANUP · LISTENERS

SUMMARY · READ-ONLY REVIEW

From Asterisk to a UDP port

1.1 The hook in rpt.conf

Enabling the receive feed for a node writes two lines into that node's block in rpt.conf: a note saying the line sends the node's audio to Node Talker, and an `outstreamcmd` line. The command runs the tool's own feed mode with Python, and passes the UDP port. The code refuses to add the hook if the node already has an `outstreamcmd` that is not Node Talker's (for example a Broadcastify feed), since a node takes only one.

1.2 The feed process

The feed process reads standard input in blocks of up to 4096 bytes and sends each block as one UDP datagram to 127.0.0.1 on the chosen port. It does no processing. It stops when the input ends or cannot be read. A failed send is ignored, so the feed keeps reading even when Node Talker is not listening. A restart of Asterisk is needed after the hook is added or changed.

1.3 The listening side

Node Talker binds a UDP socket to 127.0.0.1 on the chosen port (address reuse on), so only programs on the node can send to it, and reads datagrams of up to 8192 bytes. If the port cannot be bound, it records the error (shown on the page), logs it, and tries again every 10 seconds. After a receive error it waits one second and binds again.

Frames and the queue

SETTING	VALUE	MEANING
Frame	160 samples, 320 bytes, 20 ms	One unit of audio, 16-bit PCM at 8 kHz
Input queue limit	25 frames (0.5 s)	Above this, the oldest frames are dropped and counted as over-limit drops
Input trim point	10 frames (0.2 s)	When the pump falls behind, one frame a tick is dropped and counted as a trim drop
Listener queue	150 frames (3 s)	Each listener's queue; a full queue is counted as an overrun
Active level	-50 dBFS	A frame louder than this counts as activity
Activity hold	0.6 s	How long the channel stays "active" after the last loud frame

2.1 The pump

A paced loop runs a tick every 20 ms. If the loop falls more than half a second behind, it restarts its clock instead of rushing to catch up, and an error in one tick is logged and does not stop the loop. Each tick takes one frame off the queue, measures its level, and checks for transmit activity. It then runs the cleanup stage (Part 3) if any listener is connected or a tone test is recording. Without listeners, the tick only keeps the level and burst counts up to date, so an idle node costs little.

2.2 Bursts and kerchunks

A burst is a run of loud frames (above -50 dBFS, with gaps shorter than 0.6 s). A burst that starts while Node Talker is keying or a tone test is running, or within one second after it unkeys, is marked as the node's own transmission and is not counted. Otherwise the burst is closed when 0.6 s passes without a loud frame, and its length decides what happens:

- **Busy.** The channel counts as busy only after a burst has lasted 0.4 s. A shorter burst does not make the node busy, so it does not block a key-up.
- **Kerchunk log.** A burst of at least 0.1 s is recorded as a row of the kerchunk log (the newest 2000 rows are kept), and is marked as a kerchunk when it is shorter than the threshold. The log is stored as `kc` rows in the Suite Log file (`/run/asl_dvs/suitelog.jsonl`, the same file instmon's log collection uses), and these rows are written even when no collection is running. That folder is cleared at each reboot, so the kerchunk log and its learned threshold do not survive a reboot. An older `nodetalker_bursts.csv`, if present, is imported once and renamed with `.imported`. The threshold starts at 0.4 s. It is learned from bursts that the operator has labelled "kerchunk" or "real" on the page: it needs at least 6 labelled bursts of both kinds, is the cut with the fewest mistakes, and is kept between 0.15 s and 0.8 s. If more than a third of the labelled bursts would be misjudged, or the cut is above 0.8 s, the learned value is not used.
- **Kerchunk test.** A timed test (15 minutes by default, 1 to 60) lets the operator key up the node as usual and press Mark each time. The burst that ended in the last 5 seconds, or else the next burst, is kept as a kerchunk. The test can be stopped early, and the log can be reset, which puts the threshold back to 0.4 s.

2.3 The node's key-up state

When the node's key-up state is available from Asterisk, it replaces the audio level as the source of "busy" and of the kerchunk test. A watcher logs in to the Asterisk manager (127.0.0.1, port 5038) with the login found in `/etc/asterisk/manager.conf` and asks every 0.25 s for the main node's status (`RptStatus XStat`), reading `RPT_RXKEYED`. It runs only while it is wanted: while the node is keyed, and for 20 s after the last loud frame or key-up request.

- With the watcher working, the channel is busy while the node is keyed, and for 0.6 s after it unkeys, as well as while audio is heard. A kerchunk is a key-up shorter than 3 s whose audio is shorter than the threshold.
- The first reading after a login is treated as partial: a key-up that was already in progress is not recorded.
- If there is no login in the file, Asterisk refuses it, or the connection fails, the log says "kerchunks and busy use audio again", and the audio rules above apply. After a refused login the watcher does not retry the same login.

2.4 Delay for kerchunks

The setting `kerchunk_delay_ms` holds back all audio by a fixed time. It is 0 (off) or 100 to 1000 ms. When it is on, bursts that were marked as short kerchunks are dropped before they reach listeners, so the listener does not hear them.

Filter, gain, AGC and gate

The cleanup stage works on each frame in this order. Every stage can be switched off, and when all are off the stage only applies the RX gain.

3.1 High-pass filter

When `rx_hpf` is on, two cascaded biquad sections form a fourth-order high-pass at 300 Hz. It removes low-frequency rumble. Its filter memory is kept between frames, so there is no click at each frame edge.

3.2 Notch filter

When `rx_notch_hz` is set (60 to 300 Hz), a narrow notch removes one tone, usually a CTCSS tone that rides on the audio. The notch has a Q of 25 at an 8 kHz sample rate.

3.3 Automatic gain (AGC)

When `rx_agc` is on, the gain follows the speech level toward a target of -20 dBFS. The gain can range from -6 dB to +18 dB. It rises slowly (0.06 dB a frame) and falls faster (up to 1.5 dB a frame). After a long idle period, it moves quickly for 25 frames to catch up with a new talker.

The fixed RX gain (`rx_gain_db`, -12 to +12 dB) is added on top of the AGC gain, when the level settings are active (Part 4).

3.4 Gate

The gate (`rx_gate`) decides whether a frame is passed or silenced. It has three settings:

SETTING	MEANING	CLOSE BELOW	HOLD
0	Off, every frame passes	n/a	n/a
1 (default)	Light: closes after the level stays below -55 dBFS for 20 frames (0.4 s), or when flat loud noise (above -24 dBFS, with little sample-to-sample correlation) lasts 4 frames	-55 dBFS	20 frames
2	Strong: closes after the level stays below -48 dBFS for 12 frames (0.24 s), or when flat noise above -30 dBFS lasts 3 frames	-48 dBFS	12 frames

The gate looks four frames ahead (80 ms), so it can open before the speech starts and keep the first syllables. When it closes, the audio is faded out, and when it opens, it is faded in. The noise rule is what catches a squelch tail: loud but flat, with no speech-like correlation. The page calls setting 1 "Light" and setting 2 "Strong".

3.5 Quiet floor and limiting

When a frame is quieter than -65 dBFS, the filters are reset to silence, and the gate is driven as though the channel were empty. After 150 idle frames (3 s), any held audio is cleared. A gain that would push a sample above 29000 is passed through a soft curve, so loud peaks round off rather than clip hard.

Each listener gets its own queue

4.1 Delivery

After the cleanup stage, each frame is sent to every listener's queue. A listener that asked for 16-bit PCM (`fmt=s16`) gets the frame as it is. Any other listener gets it as u-law, which halves the data. Silence is sent as a pre-built u-law silence frame. The stream is the route `/api/listen` and answers with `application/octet-stream` and an `X-Audio-Format` header: `mulaw; rate=8000; channels=1` or `s16le; rate=8000; channels=1`. It needs a login unless `public_listen` is on.

4.2 Joining

A listener connects over HTTP and is given a slot. Up to `max_listeners` (default 3) listeners may connect. A request beyond that is refused with 503 and a message that every slot is in use. A client name is recorded from the request's user agent, so the health reports can say what kind of device is listening.

4.3 Slow listeners

Each listener's socket send buffer is 64 KB. A listener that falls behind has its queue filled, and frames are dropped for that listener only. The other listeners are not held back.

A write that takes more than 15 seconds ends the listener ("write_timeout"). A listener also ends when the client closes, and when the audio pump has not ticked for 5 seconds ("pump_stopped"). Each connect and disconnect is noted, with the reason and the time connected.

The health readings

Node Talker keeps a health record of the receive path in one-minute buckets (the last 120 minutes are kept in memory). The page's verdict looks at the last 5 minutes. A bucket that saw packets, link loss or listener underruns is also passed to the suite log (instmon's log collection), when that is switched on. The bucket notes:

- **Gaps and jitter** between packets from the feed (a gap is more than 60 ms), and how late the pump runs (more than 60 ms late is counted).
- **Drops**, split into input-queue overflow, trim and listener overruns.
- **Sends to listeners**: slow writes (0.25 s or more), audio left unsent in a listener's socket (reported from 100 ms, a stall from 500 ms).
- **IAX link statistics** for the node's links outside the LAN (loss, dropped, out of order, jitter, round trip), and the Pi's own Wi-Fi link where it can be read (signal, retries, failures).
- **Client reports** from the phone or browser (underruns, queue resets, silence), tagged android, browser or other.

5.1 The verdict

The verdict has six rows, one for each place audio can be lost, and then one summary state:

ROW	BAD WHEN
Outside the LAN (IAX links)	Any lost or dropped frames, or jitter of 60 ms or more
Audio arriving at the Pi	Any gap over 60 ms
This Pi	Input queue overflowed, pump ran late, under-voltage or throttling, or CPU at 90 % or more
This Pi's Wi-Fi	Signal worse than -80 dBm; retries of 75 % of packets, or 30 % and 20 points above the usual rate; failed sends of 5 % (or three times the usual rate); or any beacon loss, link blip, drop, chip stall, rejoin or roam. The row is information only when Wi-Fi is not connected (the Pi may be on Ethernet)
Pi to listener	Audio waited unsent, slow sends, or listener queue overruns
Phone / browser	Underruns or queue resets reported by the client

The summary state is chosen in this order: **idle** (no audio in the window), **pi** (the Pi row is bad), **wifi** (the Pi's Wi-Fi is bad while audio was lost or late), **network** (the IAX or arrival rows are bad, so audio is late before it reaches the Pi), **listener** (only the listener rows are bad, split into the phone itself or the phone's connection), and **clean**.

What the RX path shows

6.1 Two places drop audio, on purpose

The queues drop audio when the Pi falls behind, so the listener hears a short gap rather than a growing delay. The book should tell operators that a dropout on a listener is often the Pi, not the node.

6.2 The cleanup changes what the node sends

The high-pass, the AGC and the gate all change the sound before the listener hears it. A test that compares the listener's audio with the node's raw audio will show these changes. The book should say which settings were on during a test.

6.3 Not checked

- The exact health thresholds and the verdict rules, which were read only in part.
- The phone-side listener, which uses the same stream and was not traced separately.
- Real audio quality under the gate and AGC settings on a node.